

Consumo eléctrico por franja horaria, en streaming

Un pipeline end-to-end con Apache Kafka y Apache Beam sobre lecturas desordenadas, duplicadas y tardías

Proyecto integrador · Streaming de datos y sus aplicaciones
Maestría en Inteligencia Artificial, Facultad Politécnica – UNA · Prof. Rodrigo Parra, M.Sc.
Sergio Morel · Clara Almirón · Daniel Ramírez — septiembre de 2026

Resumen

Una distribuidora eléctrica necesita **cobrar la energía a precio distinto según la hora**. Para eso hace falta saber cuánto consumió cada cliente **en cada franja tarifaria**, y ese dato hoy no existe — no por falta de sistemas, sino por una restricción física: los medidores de una cabina comparten un bus RS-485 y se leen **en secuencia**, así que una ronda completa tarda entre 6 y 60 minutos y **una lectura casi nunca cae sobre el borde de una franja**.

Construimos un pipeline de streaming que produce ese dato: **Kafka → Beam sobre Flink → Kafka**, con un simulador que ocupa el lugar del concentrador e **inyecta fallas a propósito** —pedidos corridos, cabinas caídas, tramas truncadas, duplicados y ráfagas tardías—. El pipeline trabaja en **tiempo de evento**, deduplica con estado por medidor, diferencia el contador acumulado en consumos por intervalo, reparte cada intervalo entre las franjas que toca y escribe un resultado **idempotente** por celda `medidor|fecha|franja`.

El resultado no es solo el consumo: es el consumo con su margen de error. Como los intervalos cruzan bordes de franja siempre, hay que repartir suponiendo potencia constante, y cada celda declara cuánto de ella se midió y cuánto se estimó. En la demostración, una lectura tardía que cae justo sobre el borde revela que la interpolación se había equivocado en **0,100 kWh sobre 5,500** — sin que el total cambie, porque medir con más detalle no crea ni destruye energía.

De ahí sale la conclusión que el sistema habilita y que no estaba en la consigna: como la separación entre lecturas la fija la duración de la ronda, y la ronda la fija sobre todo **la tasa de fallas** y no la velocidad del enlace, **para cobrar por franja horaria conviene más mejorar la confiabilidad de la recolección que acelerarla**. Es una decisión de inversión, y este pipeline la puede medir.

El ciclo termina en un **tablero en marimó** que lee el tópico derivado haciendo *upsert*, con un inyector que publica a mano un duplicado, una tardía o un reseteo y deja ver en vivo cómo responde el pipeline.

Lo respaldan **116 pruebas** —incluidas las de `TestStream`, que son las únicas que permiten probar comportamiento tardío de forma determinista— y **seis corridas completas sobre Kafka y Flink en cuatro máquinas**, que reproducen exactamente el mismo resultado que el runner local. Dos de ellas las hicieron personas que no escribieron el código, una de ellas **ajena al equipo** y siguiendo solo el `README.md`. Dos errores de doble conteo aparecieron en el camino, y ninguno de los dos se habría visto con datos ideales: es el argumento del enunciado —una corrida feliz no es evidencia— comprobado sobre nuestro propio código.

1. El problema y para quién

Una distribuidora eléctrica necesita **facturar a precio diferenciado según la hora**: no cuesta lo mismo un kWh en hora punta que de madrugada. Para eso hace falta saber cuánta energía consumió cada cliente **en cada franja**, y ahí empiezan las dificultades.

1.1 Por qué el dato no está disponible

Los medidores de este parque **no transmiten por su cuenta ni guardan una serie histórica**: responden con el valor actual de sus contadores cuando se les pregunta. Tres hechos de esa recolección determinan todo lo demás:

Hecho	Consecuencia
El registro <code>15.8.0</code> es un contador acumulado	El consumo no viene en el dato: se obtiene restando dos lecturas
Los medidores de una cabina comparten un bus RS-485 y se leen en secuencia	Ninguna lectura cae justo en el borde de una franja; la ronda dura de 6 a 60 minutos
El concentrador pierde enlace y publica después lo que juntó	Las lecturas llegan tardías, fuera de orden y en ráfagas

Hoy el sistema pide **una vez por día**, lo que alcanza para facturar el consumo diario y no para discriminar por franja. Habilitar la tarifa horaria exige pedir en cada borde, y ese es el cambio que este trabajo modela.

1.2 El giro: el tiempo de evento es dinero

En la mayoría de los pipelines, la distinción entre *tiempo de evento* y *tiempo de procesamiento* es una sutileza técnica. **Acá no**. Una lectura atribuida a la franja equivocada se factura a un precio equivocado, y el error no es aleatorio: se concentra en las cabinas grandes y de peor enlace, que son las que más tardan en completar una ronda.

Eso convierte una decisión de diseño en un requisito de negocio, y es la razón de que toda la política temporal del sistema esté justificada y no simplemente elegida.

1.3 Quién usa el resultado

Dos consumidores del mismo tópico, con necesidades opuestas:

	Tablero operativo	Facturación
Qué lee	Todas las revisiones de cada celda	Un solo valor por celda
Cuándo	Continuamente	Pasado <code>ventana_fin + 36 h</code>
Qué tolera	Que el número cambie mientras la ventana no converge	Nada: necesita un valor estable
Qué habilita	Ver la demanda por franja mientras el día transcurre	Emitir la factura con el precio correcto por franja

Esa diferencia es la que justifica el modo **acumulativo** con salida por *upsert*: cada pane es la revisión completa de la celda, así que el tablero puede mostrar el último y la facturación puede leer una sola vez sin reconstruir nada.

El tablero está construido; la facturación no. El tablero es una aplicación marimo en `tablero/` y se describe en las secciones 2 y 6.5. La facturación queda como el segundo lector que el contrato contempla: su patrón de lectura está definido, pero implementarla no agrega nada que el tablero no demuestre ya sobre el mismo tópico.

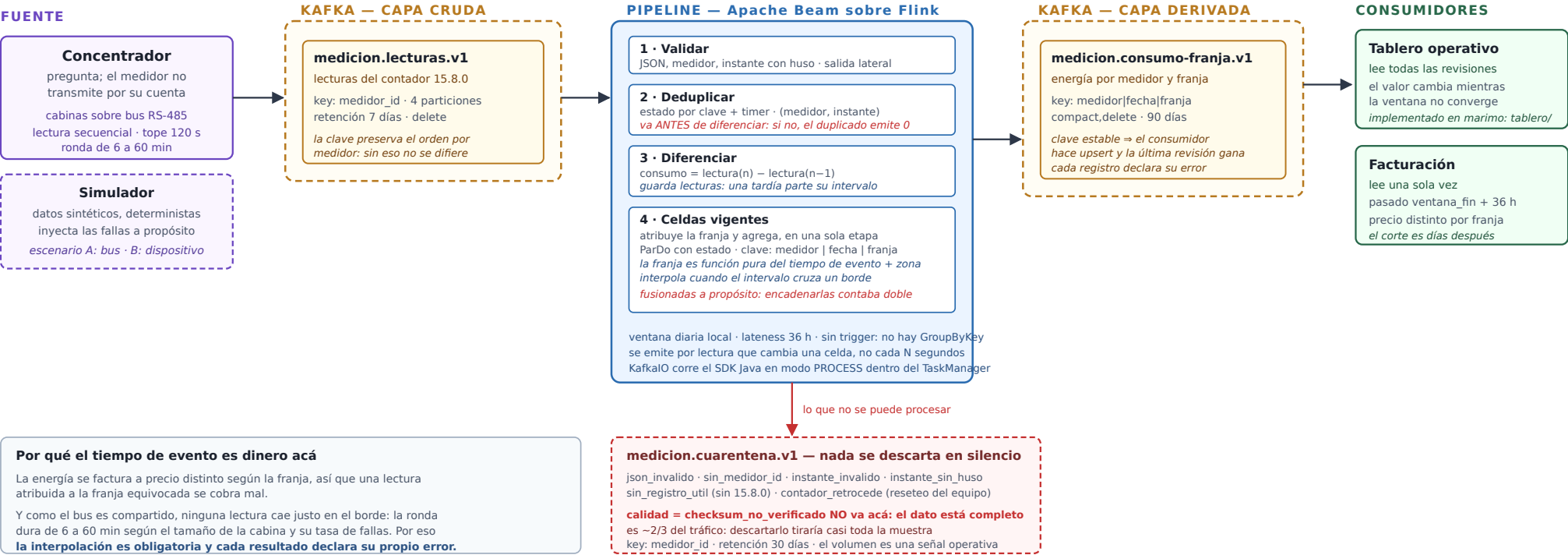
1.4 Qué mide el sistema, además del consumo

Un resultado que el trabajo produce y que no estaba en la consigna: **cada registro de salida declara su propio error de atribución**, derivado de la separación entre las dos lecturas que rodean el borde de la franja. El sistema no solo entrega un número, entrega cuánto se puede confiar en él — y eso permite cuantificar cuánto costaría, en dinero mal facturado, la arquitectura de recolección actual frente a una con un dispositivo por medidor.

2. Arquitectura

Consumo eléctrico por franja horaria — arquitectura

Proyecto integrador · Streaming de datos y sus aplicaciones · FPUNA. Flujo: concentrador → log crudo → pipeline → log derivado → consumidores.



El medidor entrega un contador acumulado (OBIS 15.8.0): el consumo no viene en el dato, se obtiene restando dos lecturas. Todos los datos son sintéticos.
Escenario B — un dispositivo de lectura por medidor, sin bus compartido — corre el MISMO pipeline y permite medir cuánto error elimina esa arquitectura.

Arquitectura del sistema: concentrador, log crudo, pipeline, log derivado y consumidores.

El flujo es **concentrador** → **log crudo** → **pipeline** → **log derivado** → **consumidores**, con una salida lateral a cuarentena. Cada componente está donde está por una razón:

El simulador ocupa el lugar del concentrador. Genera datos sintéticos y deterministas, y existe para **inyectar fallas a propósito**: pedidos que se corren, cabinas que caen enteras, tramas truncadas, duplicados y ráfagas tardías. Un simulador que se porta bien no sirve para demostrar que el pipeline tolera lo que tiene que tolerar.

Kafka en dos capas. El tópico crudo conserva las lecturas tal como llegaron, claveadas por medidor para que su orden se preserve — sin ese orden, la etapa de diferenciación no puede restar lecturas consecutivas. El tópico derivado lleva el resultado, claveado por la celda que identifica, de modo que recalcular reemplace en lugar de duplicar.

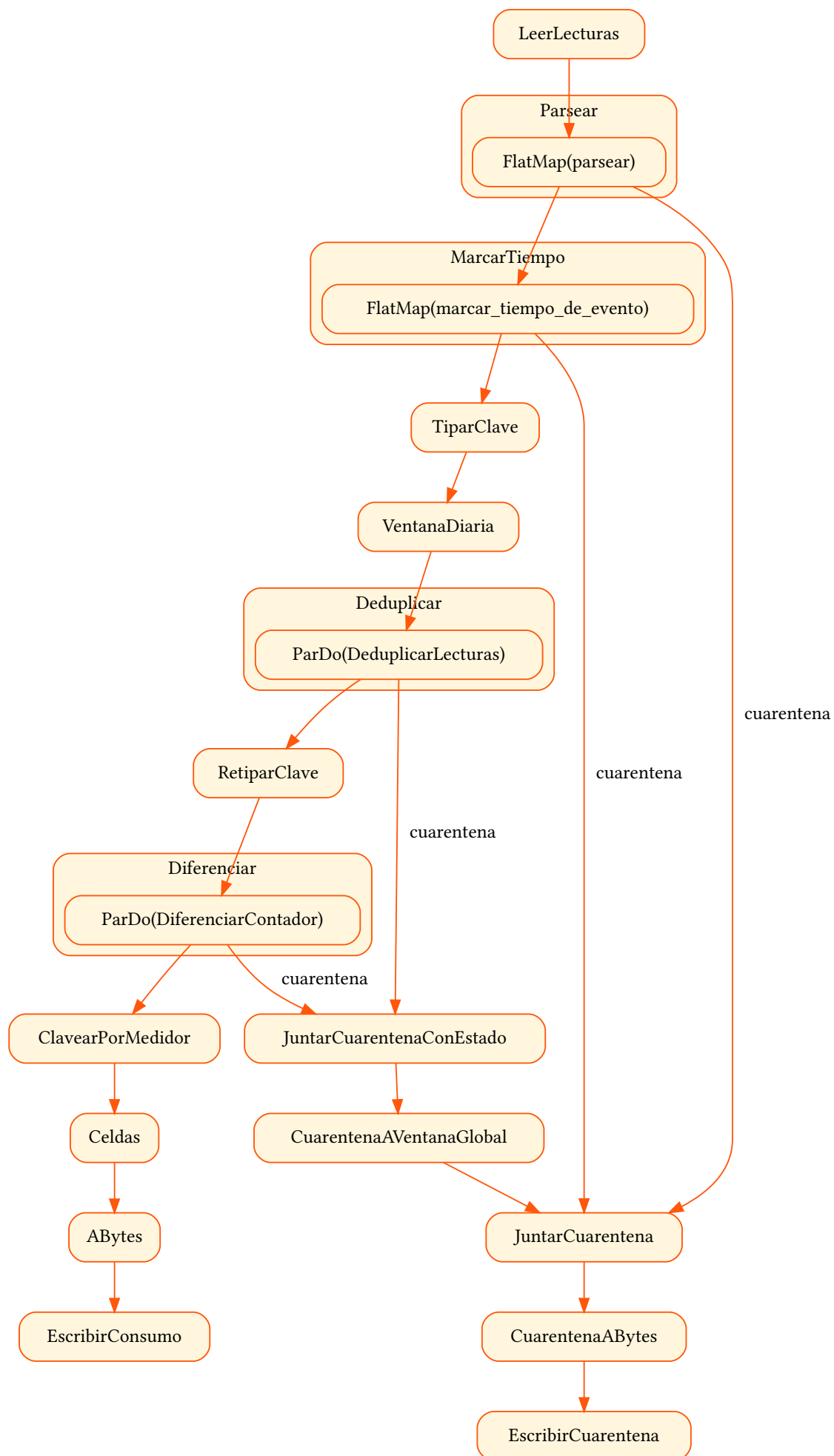
El pipeline sobre Beam y Flink hace cinco cosas en orden, y **el orden no es intercambiable**: la deduplicación va antes de la diferenciación porque, si no, un duplicado se restaría contra sí mismo y produciría un consumo de cero que, con salida por *upsert*, pisaría el valor correcto.

La cuarentena recibe todo lo que no se puede procesar, contado y con su motivo. Su volumen es una señal operativa: si sube en una zona, el problema es la cobertura de red y no el pipeline.

El tablero cierra el ciclo del dato —producir, procesar, analizar—. Lee el tópico derivado desde el offset 0 en cada actualización y se queda con la última revisión de cada celda, así que el *upsert* y el reproceso que el contrato declara se ejecutan a la vista. Valida el contrato de salida antes de aceptar un mensaje, y lo que no lo cumple lo cuenta y lo informa. Carga el calendario tarifario con el mismo `cargar_calendario` que el pipeline, porque la cota del error de atribución la calcula el consumidor (sección 4.6). `marimo` se resuelve con `uv run --with marimo`, así que ni el entorno del proyecto ni la imagen de Docker cargan con una dependencia de visualización.

2.1 El grafo que corre de verdad

El diagrama de arriba se mantiene a mano y muestra los componentes. El de abajo no: lo dibuja Beam a partir del mismo `construir + cadena` que se ejecuta en producción, con `uv run python -m pipeline.grafico`. Por eso no puede quedar desactualizado respecto del código, que es el riesgo de todo diagrama de pipeline.



Se lee de arriba hacia abajo: el camino principal a la izquierda y, a la derecha, **las cuatro ramas de cuarentena** convergiendo en un solo tópico. Las cajas `LeerLecturas`, `EscribirConsumo` y `EscribirCuarentena` están cerradas a propósito: por dentro `KafkaIO` despliega una docena de pasos del SDK de Java que no dicen nada sobre este pipeline.

Cada etapa por separado —con su entrada, su proceso, su estado, sus temporizadores y los motivos por los que manda algo a cuarentena— está en `docs/diagramas/etapas/`, una lámina por etapa. Las secciones 5 y 6 explican las que tienen estado.

Y para ver **en qué se convierte un dato concreto** en cada punto del recorrido, de la lectura del medidor a la celda que lee el consumidor, está `docs/diagramas/recorrido/`. Sigue la misma lectura de la sección 6.1, así que los números son los de la demostración.

2.2 El detalle que condiciona el despliegue

KafkaIO no es una librería Python: es una transformación *cross-language* cuyas etapas de lectura y escritura ejecuta el SDK de **Java**. La imagen de Flink empaqueta los dos SDK y los corre en modo `PROCESS` dentro del `TaskManager`, lo que evita tener que darle al contenedor acceso al demonio de Docker. Está documentado en `infra/README.md`.

El job server, que prepara los artefactos de cada trabajo, **no libera memoria entre trabajos:** recorriendo el README entero pasa de 400 MB a 1,75 GB. Su límite está en 1800m. Con el valor anterior, 900 MB, moría por OOM en la prueba de humo, que es el cuarto trabajo que sirve. Lo encontró la verificación externa (sección 6.6), y no nuestra automatización, porque la automatización no deja un trabajo de streaming corriendo mientras lanza los demás. El stack completo llega a 8,4 GB, y el README pide 12 GB libres.

2.3 Observabilidad

El enunciado pide «logs o métricas suficientes para observar producción, consumo, procesamiento y errores». El pipeline cuenta con la API de métricas de Beam lo que pasa en el camino caliente:

Métrica	Qué deja ver
<code>cuarentena/total</code> y <code>cuarentena/<motivo></code>	Cuánto se rechaza y por qué . El total no alcanza: no distingue un concentrador desincronizado de un tópico con mensajes de otro productor
<code>dedup/duplicados_descartados</code>	El trabajo que la deduplicación hace sin mandar nada a cuarentena, porque un duplicado no es un error. Sin este contador sería invisible
<code>diferenciacion/consumos_emitidos</code>	Intervalos que salen de restar lecturas
<code>atribucion/separacion_minutos</code>	Distribución —mínimo, máximo, media— de la separación entre lecturas, que es la métrica central del proyecto : si la media sube, las rondas se alargaron y el reparto por franja se volvió más grueso
<code>salida/celdas_emitidas</code> y <code>salida/celdas_indeterminadas</code>	Revisiones publicadas, y cuántas cayeron por encima del umbral de 90 min

Hoy no se ven en la interfaz de Flink, y conviene decirlo porque se llegó a escribir lo contrario. El `flink-conf.yaml` no configura ningún `metrics reporter` y el runner portable no las publica solo: `/jobs/<id>/accumulators` y `/jobs/<id>/vertices/<id>/metrics` salen vacíos. Se consultan con `PipelineResult.metrics().query()`, y exponerlas en la interfaz es configuración de Flink (sección 7.3).

Lo que sí se observa sin nada más: la interfaz de Flink en `localhost:8081`, con los registros que entran y salen de cada vértice; los offsets y el lag de cada grupo con `kafka-consumer-groups`; el log de arranque del pipeline, que informa el calendario cargado y el desplazamiento de la ventana; y el tablero, que muestra la cuarentena por motivo al lado de las celdas.

3. Contrato de eventos y topología de Kafka

3.1 El evento de entrada

Tópico `medicion.lecturas.v1` · **clave** `medidor_id` · **4 particiones**.

```
{
  "schema_version": 1,
  "event_id": "0c12196f5ef41988",
  "medidor_id": "MED-0002-000",
  "cabina_id": "CAB-0002",
  "lote_id": "LOTE-CAB-0002-0000",
  "secuencia": 0,
  "instante_lectura": "2026-09-22T00:00:18-03:00",
  "registros": [
    { "obis": "15.8.0", "naturaleza": "acumulado", "valor": 10898.124, "unidad": "kWh" }
  ],
  "calidad": "ok",
  "publicado_at": "2026-09-22T00:00:19.269-03:00"
}
```

Tres campos merecen explicación porque no son obvios:

event_id es **determinista**, `sha256("<medidor_id>|<instante_lectura>")[:16]`. Si fuera aleatorio, un reintento de publicación produciría un id distinto y el duplicado dejaría de ser reconocible. Es la mitad de la clave de deduplicación.

cabina_id no es **decorativo**. La duración de la ronda —y por lo tanto el error de atribución— es una propiedad **de la cabina**, porque sus medidores comparten un bus RS-485 y se leen en secuencia. Sin este campo no se distingue «un medidor no contesta» de «se cayó la cabina entera».

publicado_at **contra** **instante_lectura** da el retraso de publicación, que es el insumo de la observabilidad y lo que justifica el valor de la lateness.

Los registros van como **lista de objetos y no como mapa código → valor**, porque la unidad y la naturaleza son propiedad del registro y no de la lectura: `15.8.0` viene en kWh, pero un registro de potencia vendría en kW. Con un mapa, esa tabla viviría hardcodeda en el consumidor. Lo que la lista pierde —un mapa impide por estructura que un código se repita— pasa a ser una regla que el validador hace cumplir.

3.2 Por qué la clave es el medidor y no la cabina

El orden que este caso necesita es **por medidor**, y es un requisito duro: la diferenciación resta lecturas consecutivas del mismo equipo, así que necesita verlas en secuencia. Kafka preserva el orden dentro de una partición, y con esta clave todas las lecturas de un medidor caen siempre en la misma.

cabina_id agruparía la ronda, que es tentador, pero las cabinas van de 1 a 199 medidores: produciría particiones desbalanceadas en un factor de **200**. Con **medidor_id** el reparto es parejo por construcción.

3.3 Cuatro particiones, y por qué el criterio no es el throughput

Aun el caso grande —100.000 medidores en rondas continuas— son unos 9,6 millones de mensajes por día, que para un broker es poco. Lo que fija el número es otra cosa:

1. **El paralelismo útil está acotado por `min(particiones, slots, claves)`**. El stack corre 2 TaskManagers con 2 slots cada uno, y cada trabajo pide paralelismo 2: el pipeline de streaming ocupa dos slots y no los suelta, y el recorrido de punta a punta necesita los otros dos. Con 4 particiones, un solo trabajo puede llegar a paralelismo 4 sin reparticionar, que es una operación que rompe el orden por clave.
2. **El estado vive por clave, no por partición**, así que agregar particiones no alivia memoria: solo reparte.
3. **Más particiones alargan la recuperación**, porque releer una partición es serial.

3.4 Tópicos, retención y evolución del esquema

Tópico	Para qué	Clave	Retención
<code>medicion.lecturas.v1</code>	Lecturas crudas	<code>medidor_id</code>	<code>delete</code> , 7 días
<code>medicion.consumo-franja.v1</code>	Resultado por medidor, día y franja	<code>medidor fecha franja</code>	<code>compact,delete</code> , 90 días
<code>medicion.cuarentena.v1</code>	Todo lo rechazado, con su motivo	<code>medidor_id</code> si se conoce	<code>delete</code> , 30 días

La convención es `<dominio>.<sustantivo>.<mayor>`, y la regla de evolución tiene dos ramas. Un **cambio compatible** —un campo opcional, un registro OBIS más en la lista— no cambia nada, porque un consumidor que solo mira `15.8.0` no se entera. Un **cambio incompatible** —quitar o renombrar un campo, cambiar un tipo o su significado— crea un tópico `.v2`, y los dos conviven mientras dure la transición.

schema_version viaja en **tres lugares** y cada uno tiene su razón: en el nombre del tópico, para que un consumidor incompatible no se suscriba siquiera; en un header, para decidir sin deserializar; y en el cuerpo, para que el mensaje sea autodescriptivo cuando se lo mira suelto, en un archivo o en la cuarentena.

En la salida, `compact,delete` es una combinación deliberada: **compact solo nunca borra una clave**, y como la clave incluye la fecha local, el espacio de claves crecería todos los días para siempre. La política combinada conserva el último valor de cada clave y deja caducar las viejas.

3.5 La bandera de calidad que no significa corrupción

Es el hallazgo más fuerte del análisis del dominio, y el que más plata cambia.

Valor	Qué significa	Qué hace el pipeline
<code>ok</code>	Trama completa y verificada	Procesa
<code>checksum_no_verificado</code>	La trama llegó entera , pero no se pudo verificar su carácter de control	Procesa , y lo cuenta
<code>truncada</code>	La trama se cortó	Cuarentena

La lectura ingenua de `checksum_no_verificado` es «dato sospechoso, descartar». Sería un error caro: **es del orden de dos tercios del tráfico** —medido sobre el simulador, 63,2 % con 8 cabinas— y los datos están completos. Descartarlos tiraría casi toda la muestra. No indica corrupción sino un equipo cuyo cálculo del carácter de control difiere del que el concentrador espera, que es una propiedad del **modelo de medidor**, no del estado de la trama.

Y *truncada* **no alcanza como defensa**: una trama cortada en medio de un número —014380.81 → 01438— sigue siendo un número válido y no siempre se marca. Por eso hay una segunda defensa aguas abajo, en la diferenciación: un contador no baja (sección 5.6).

3.6 El registro de salida

Tópico `medicion.consumo-franja.v1` · **clave** `medidor_id|fecha_local|franja`.

La clave **es el contrato**: es estable a través de todas las revisiones de una misma celda, todas van a la misma partición, se leen en orden y la última gana. La semántica del consumidor es **upsert, nunca insert**. No incluye `cabina_id` aunque el campo viaje en el valor, porque un medidor podría cambiar de cabina y la identidad del resultado no debe depender de eso.

Cada registro **declara su propia incertidumbre**, que es lo que lo hace honesto: *interpolada* dice si algún borde se estimó en lugar de medirse, *separacion_maxima_minutos* da la cota del error de atribución, y *minutos_cubiertos* contra la duración de la franja distingue «consumió poco» de «todavía no llegó todo».

Y hay un límite a lo que se estima. Repartir por interpolación supone potencia constante durante el intervalo, y eso deja de ser defendible cuando el intervalo dura horas: en *punta* sabemos que el consumo no es uniforme — es la razón de que la franja exista. Por encima de **90 minutos** la energía de ese cruce no se reparte: la celda queda *indeterminada* y *minutos_indeterminados* dice cuánto quedó sin cubrir. El resto de la celda se conserva.

El umbral no se eligió por gusto. La duración de los intervalos que cruzan un borde tiene **dos modas**: abajo de 90 minutos está la ronda normal, aun una lenta, con el 79 % de los casos; arriba de 120 está la cabina caída, con el 20 %. Entre medio hay 13 casos de 1.146. El umbral va en ese valle, porque a cada lado hay un fenómeno distinto — y cortar más abajo marcaría indeterminado el 42 % de los cruces, casi todos rondas que funcionaron bien.

La cobertura se mide en **minutos** y no en cantidad de intervalos, que sería la forma habitual. Con *readout* ese número no existe: no hay grilla fija de medición sino rondas continuas cuya duración depende del tamaño de la cabina, así que no hay un total de intervalos «esperados» contra el cual comparar.

4. Tiempo de evento, ventanas y datos tardíos

4.1 Cuál es el tiempo de evento, y quién lo pone

El record de Kafka trae **cuándo se publicó**; *instante_lectura* trae **cuándo se midió**. Entre los dos puede haber horas —es justamente el retraso que justifica la *lateness*— y ventanear por el de publicación metería consumo en el día equivocado. El pipeline asigna el timestamp explícitamente después de parsear.

El instante lo pone el **concentrador**, no el medidor, y esa es una decisión del dominio y no una comodidad: el modo *readout* no devuelve timestamp, y muchos equipos tienen el reloj mal configurado o desactualizado. El concentrador sella al recibir la respuesta, lo que traslada el problema de «miles de relojes dudosos» a «unos pocos relojes que se pueden sincronizar».

4.2 La ventana diaria, y por qué hay que desplazarla

Ventana fija de un día, alineada a la medianoche local.

Beam ventanea sobre el instante absoluto, así que una `FixedWindows(1 día)` sin desplazar cortaría a medianoche UTC — **las 21:00 en Asunción, en pleno horario de punta**. El corte caería en el medio de la franja más cara. Se corrige con un desplazamiento de 3 h, que el pipeline calcula del propio calendario en lugar de tenerlo escrito a mano: si alguien cambia la zona en la configuración, el desplazamiento la sigue.

La franja no es una ventana, y conviene decir por qué se evaluó y se descartó. Una franja es función pura del tiempo de evento, así que no necesita agrupamiento: se calcula y **viaja en la clave**. Modelarla como ventana obligaría a redefinir el ventaneo cada vez que cambiara el calendario tarifario, y el calendario es configuración.

4.3 Las 36 horas de lateness

Alternativa	Por qué no
24 h	Es exactamente la cadencia de recolección, sin margen: cualquier corte que dure un poco más pierde datos
48 h	Duplica el estado sin evidencia de que haga falta
36 h	Cubre un día entero de caída de enlace con media jornada de margen

El máximo retraso medido en el corpus fue de **3,97 h**, un orden de magnitud por debajo. La holgura no es por incertidumbre sobre el caso típico sino por el caso raro: una cabina que queda incomunicada un día entero.

Ese número es también el horizonte de la deduplicación (sección 5.2) y el tiempo que vive el estado. No es coincidencia: si el estado expirara antes, un tardío legítimo volvería a parecer nuevo.

4.4 Triggers, panes y acumulación

El pipeline no configura triggers, y conviene explicar por qué, porque es consecuencia directa de la corrección de la sección 5.4.

Los triggers disparan en un `GroupByKey` o un `Combine`. Cuando las dos etapas de agregación se fusionaron en una sola con estado —para que dejaran de contar doble—, **dejó de haber agregación en el grafo**: lo que queda son `ParDo` encadenados. Un trigger configurado ahí no se dispararía nunca, y dejarlo escrito sugeriría un comportamiento que no ocurre.

Lo que la ventana sí aporta sin trigger:

Qué	Para qué
<code>FixedWindows(1 día, desplazada 3 h)</code>	Da el <code>window.end</code> contra el que se programan los timers, y alinea el día al calendario local
<code>allowed_lateness = 36 h</code>	Fija cuándo expira el estado por medidor, que es lo que acota la memoria

El efecto práctico es mejor que el de un disparo periódico: la salida se emite **por cada lectura que cambia una celda**. El tablero no espera a un reloj, se mueve cuando hay información nueva — y cuando no la hay, no escribe nada.

Lo que se resigna: emitir por llegada significa que la ráfaga de una cabina que vuelve de una caída produce una escritura por lectura. En producción convendría agrupar esos disparos y demorar la corrección unos minutos — algo que a la facturación no le cambia nada, porque se factura días después. Se elige la versión por evento **para que la corrección sea visible en la demostración**, y queda declarado como límite conocido.

Nada anuncia que una revisión es la última. Después del último tardío simplemente no se emite más. La finalidad la deduce el consumidor cuando su reloj pasa `fin_de_ventana + 36 h`, y por eso hay dos lectores del mismo tópico con patrones distintos: el tablero lee todas las revisiones y muestra un valor que cambia; la facturación lee una sola vez, pasado ese horizonte.

4.5 La validación va antes del watermark

El orden es una decisión y no un detalle: **un evento se valida antes de participar del avance del watermark**. Un solo concentrador con el reloj adelantado arrastraría el watermark hacia el futuro y haría que Beam descartara por tardías las lecturas legítimas **de todos los demás medidores**. Un dato malo pasaría de arruinar su propia celda a arruinar la ventana entera.

Por eso una lectura sin offset horario va a cuarentena en lugar de interpretarse en la zona del proceso: si se asumiera la zona local del worker, la franja atribuida dependería de en qué máquina corre el pipeline.

4.6 El error de atribución, que es el resultado central

```
error_atribucion = separacion_maxima_minutos / duracion_franja_minutos
```

No es un campo del mensaje: lo calcula el consumidor. El pipeline emite `separacion_maxima_minutos`, que es el numerador; el denominador es la duración de la franja, que sale del calendario tarifario y no del evento. Ponerlo en el mensaje obligaría a que el pipeline y el consumidor estuvieran de acuerdo sobre qué calendario rige en cada fecha, que es justamente lo que la decisión de dejar el calendario en configuración evita.

Con la franja `punta` de 4 h del calendario de ejemplo y una separación máxima de 38 min, da **15,9 %**. El número es distinto para cada medidor, porque depende del tamaño de su cabina, y en el corpus medido va de 16 min en la mediana a **249 min** en el peor caso: una cabina caída.

Lo que hay que entender de ese número es de dónde sale. Los medidores de una cabina comparten un bus RS-485 y se leen **en secuencia**, así que la separación entre dos lecturas del mismo equipo la fija la duración de la ronda. Y la duración de la ronda la fija sobre todo **la tasa de fallas**, no la velocidad del enlace: cada medidor que no contesta cuesta el tiempo de espera y los reintentos.

De ahí sale la conclusión que el sistema habilita y que no estaba en el enunciado: **para cobrar por franja horaria, mejorar la confiabilidad de la recolección vale más que acelerarla**.

5. Confiabilidad: duplicados, idempotencia y garantías

5.1 Dos reintentos que no producen lo mismo

Es la distinción de la que depende no descartar datos buenos:

Reintento	Qué produce	Quién lo maneja
De comunicación — se vuelve a pedir al medidor	Una lectura nueva , en un instante posterior. No es un duplicado	La lógica de franja, tolerando que la lectura no esté donde se la esperaba
De publicación — se vuelve a publicar a Kafka	Un duplicado real : mismo instante, mismo valor	La deduplicación

Tratar el primero como duplicado descartaría una medición legítima.

5.2 Deduplicación, y por qué va antes de diferenciar

Clave: (`medidor_id`, `instante_lectura`), que es exactamente lo que resume el `event_id` determinista. Horizonte: **36 horas**, el mismo que la `lateness` — si el estado expirara antes, un tardío legítimo volvería a parecer nuevo y se contaría dos veces.

El orden respecto de la diferenciación **no es intercambiable**:

```

1.ª vez: consumo = R2 - R1      OK
2.ª vez: consumo = R2 - R2 = 0    MAL y con upsert, ese 0 pisa el valor correcto

```

Hay una prueba con `TestStream` que fija este orden, de modo que nadie pueda invertirlo sin que la suite falle.

5.3 La salida idempotente

El pipeline **no puede evitar reintentar**: un timeout de escritura no dice si la escritura llegó. Por eso la idempotencia tiene que estar en la **forma de la salida** y no en no reintentar.

La clave `medidor|fecha|franja` identifica la **celda del resultado**, no el intento de escritura. Todos los panes de una celda comparten clave, van a la misma partición, se leen en orden y **el último gana**. Recalcular una ventana reemplaza su valor en lugar de sumar otro, y eso es lo que hace que el replay converja al mismo resultado.

5.4 El upsert no alcanza: un intervalo superado no puede seguir sumando

La clave anterior resuelve los reintentos de escritura, pero **no** el caso de la lectura tardía, y conviene separarlos porque se parecen.

Cuando llega una tardía, la diferenciación parte el intervalo que la contenía y emite las dos mitades. El intervalo grosero ya salió, y Beam Python no tiene retractaciones. El *upsert* no lo retira, porque opera sobre la **celda** y los tres intervalos caen dentro de la misma celda. Con `ACCUMULATING`, la agregación los suma a los tres:

Intervalo	Energía
08:00 → 09:00	6 kWh
08:00 → 08:30	2 kWh
08:30 → 09:00	4 kWh
Suma	12 kWh , el doble del consumo real

La regla que lo corrige: los intervalos de un medidor parten la línea de tiempo, y cada uno queda identificado por su **borde izquierdo**. Partirlo exige una lectura interior, que acerca el borde derecho — un intervalo solo puede **acortarse**. Entre varios que empiezan en el mismo instante, **el vigente es el más corto**.

Es función pura del dato, no del orden de llegada, y por eso un *replay* converge al mismo resultado. Con la regla alternativa —«el último que llegó»— haría falta un orden que en un reproceso no existe.

Y hay una segunda trampa, del mismo error. La primera corrección fue descartar los superados con un `CombinePerKey` y dejar la agregación por celda detrás. Estaba mal igual: **encadenar dos agregaciones bajo un trigger `ACCUMULATING` cuenta doble**, porque cada pane de la primera llega a la segunda como un elemento nuevo y la segunda lo suma otra vez.

Pane	Resultado
1	6 kWh OK
2	12 kWh MAL — y es el que vale, porque el último gana

Por eso las dos etapas son **una sola**, con estado y sin `GroupByKey`: `process` corre una vez por elemento, no una vez por pane. Emite el **valor absoluto** de cada celda que cambia, de modo que el destino sea un *upsert* puro.

Los dos errores los encontraron pruebas, no ejecuciones. Ninguno aparece con datos ideales: el primero necesita una lectura tardía, el segundo además necesita que la ventana dispare dos veces. Son exactamente los escenarios adversos que el enunciado pide demostrar, y la razón por la que una corrida feliz no es evidencia.

5.5 Qué garantiza el sistema, y dónde termina

Declarado por tramo, sin sobreprometer:

Tramo	Garantía	Por qué
Concentrador → Kafka	Al menos una vez	El productor declara <code>acks=all</code> e idempotencia: lo confirmado está en todas las réplicas, y sus reintentos internos no duplican ni reordenan. El que duplica es el reintento de publicación de la sección 5.1, que es otro envío
Dentro del pipeline	Efectivamente una vez dentro del horizonte de 36 h, mientras el trabajo no se reinicie	Deduplicación con estado por clave. El respaldo tendría que darlo el checkpointing de Flink, y en este despliegue no completa — ver la sección 7.2
Pipeline → salida	Efectivamente una vez en el efecto observable	El <i>upsert</i> por clave estable hace que reescribir sea ino-cuo

No se afirma exactly-once de punta a punta, y conviene decir por qué: fuera del horizonte de 36 horas la deduplicación no garantiza nada, porque el estado ya expiró. Un duplicado que llegara al tercer día se contaría de nuevo. Es una decisión consciente — mantener el estado indefinidamente no es una opción sobre una entrada no acotada — y el límite queda declarado en lugar de escondido.

5.6 Un consumo negativo nunca es válido

En el mercado modelado no hay compra de energía al usuario, así que el contador solo puede subir. Una resta negativa es un **reseteo del equipo** o una **trama truncada**, nunca una medición: va a cuarentena.

Esa regla es además la segunda defensa contra el caso más peligroso del dominio — una trama cortada en medio de un número, que deja un valor plausible que ninguna validación de formato detecta.

6. Pruebas y evidencia

El enunciado fija la vara: «una ejecución exitosa con datos ideales no es evidencia suficiente». Por eso la evidencia está partida en piezas con propósitos distintos: una historia controlada, las pruebas, el recorrido sobre el stack real, las irregularidades a pedido y la reproducción en otras máquinas.

6.1 La demostración narrada

uv run python -m pipeline.demonstracion cuenta una historia de cinco lecturas sobre **un solo medidor**, elegida para que cada paso se verifique con una resta mental. Corre con `DirectRunner` y `TestStream`, sin Docker: el tiempo se controla, así que la salida es idéntica en cualquier máquina — que es lo que la vuelve evidencia y no anécdota.

Las lecturas del contador, en kWh acumulados, rodean el borde de las 18:00 donde empieza **punta**:

Acto	Qué llega	Qué debe pasar
1	17:40 → 100,0 · 17:55 → 101,5 · 18:20 → 105,5	Dos intervalos; el segundo cruza el borde y se reparte por interpolación
2	Otra vez 17:55 → 101,5	Nada cambia. Es un reintento de publicación
3	Tardía: 18:00 → 102,4	Parte el intervalo y corrige el reparto estimado

Y lo que efectivamente sale:

Celda	Acto 1 y 2	Acto 3	Diferencia
MED-0042 2026-09-25 resto	2,300 <i>interpolado</i>	2,400 <i>medido</i>	+0,100
MED-0042 2026-09-25 punta	3,200 <i>interpolado</i>	3,100 <i>medido</i>	-0,100
Total	5,500	5,500	0

Hay tres cosas para leer ahí, y son las tres que el enunciado pide ver.

El duplicado no movió la tabla. No apareció ningún intervalo de 0 kWh pisando un valor bueno, que es lo que pasaría si se deduplicara después de diferenciar (sección 5.2).

La tardía corrigió el reparto sin cambiar el total. Medir con más detalle no crea ni destruye energía: solo cambia a qué franja se le atribuye. La columna de origen pasa de *interpolado* a *medido*, porque la lectura cayó justo sobre el borde y ya no hubo nada que estimar.

Los 0,100 kWh son el error de atribución, el número que el proyecto existe para medir. No es un defecto del pipeline: viene de que el intervalo cruzaba el borde y hubo que suponer potencia constante. La tardía es la que revela cuánto se erró.

La demostración verifica sus propias afirmaciones y devuelve código de salida, y está cubierta por la suite: si se rompe, el video que la muestra deja de ser reproducible.

6.2 Las pruebas automáticas

116 en total, y la división importa:

Suite	Cuántas	Qué fija
Simulador	34	Determinismo por semilla, también entre procesos; inyección de fallas, curva de consumo
Franjas	29	Validación del calendario, atribución, reparto por borde, conservación de la energía
Inyector del tablero	24	Que cada botón produzca un evento del contrato, y que el duplicado conserve el <code>event_id</code>
Etapas con estado	11	Duplicado, desorden, contador que retrocede, orden de las etapas, dos panes que no suman doble
Cadena completa	12	Ventana alineada al día local, cuarentena por motivo, lectura sin huso, umbral de 90 min
Demostración	3	Que la evidencia de la sección 6.1 siga saliendo como está escrita acá
Arranque	3	El punto de entrada que usa el README, que las demás no ejercitaban (sección 6.6)

Corren igual en el host con `uv run pytest` y dentro del contenedor con el perfil `pruebas`, que existe porque en una de las máquinas de verificación `uv run pytest` abortaba con un *segmentation fault* durante la colección.

Las de `TestStream` son las que no se pueden escribir de otra forma: el comportamiento tardío depende de dónde está el watermark, y con un reloj real habría que esperar y el resultado dependería de la máquina.

Dos merecen mención porque fijan decisiones que alguien podría deshacer sin darse cuenta:

- `test_el_duplicado_debe_deduplicarse_antes_de_diferenciar` — el orden de las dos etapas.
- `test_la_tardia_no_puede_contarse_dos_veces` — que el intervalo superado deje de sumar.

6.3 El recorrido completo sobre Kafka y Flink

Son dos pruebas, y la separación es deliberada.

`pipeline.humo` verifica el **cableado** con un *passthrough*: que KafkaIO levanta, que Flink acepta el trabajo, que los bytes entran y salen. Sin lógica de dominio de por medio, separa «el pipeline está mal» de «la infraestructura está mal», que son dos problemas distintos.

`pipeline.extremo_a_extremo` responde la pregunta que ninguna de las otras responde: **¿la lógica da lo mismo cuando la ejecuta Flink?** No es retórica. El runner portable serializa las funciones y el estado hacia procesos que no comparten memoria con el que arma el pipeline, y hay cosas que andan en `DirectRunner` y no allá. Siembra las mismas cinco lecturas de la sección 6.1 y exige el mismo resultado:

celda	kWh	esperado	origen
MED-0042 2026-09-25 punta	3.100	3.100	medido OK
MED-0042 2026-09-25 resto	2.400	2.400	medido OK
TOTAL	5.500	5.500	

Dos detalles que la corrida deja ver y que valen como evidencia por sí solos. La **cuarentena quedó vacía**, así que nada se perdió por el camino. Y el tópico de salida recibió **cinco mensajes para dos celdas**: son las revisiones sucesivas, una por cada lectura que cambió algo, y el *upsert* del consumidor se queda con las dos últimas. Es la semántica del contrato funcionando sobre el stack real, no sobre una maqueta.

Usa tópicos propios (`medicion.*.e2e`) para que cada corrida sea independiente: compartir los de producción hacía que leyera lo que había dejado la prueba anterior. Comandos en `infra/README.md`.

Y el replay, que es la idempotencia sobre el stack real. Después del recorrido, el servicio `repeticion` relee el tópico de entrada **desde el offset 0**, con un `group.id` nuevo y un trabajo de Flink sin estado previo, y escribe a un tópico de salida propio para que sus celdas se lean solas. Sin sembrar nada: el mismo log, procesado otra vez desde cero.

celda	kWh	esperado	origen
MED-0042 2026-09-25 punta	3.100	3.100	medido OK
MED-0042 2026-09-25 resto	2.400	2.400	medido OK
TOTAL	5.500	5.500	

Converge a las mismas dos celdas, con la cuarentena vacía. La demostración de la sección 6.1 ya hacía un replay con estado nuevo en cada acto, pero con `DirectRunner`; esto lo repite con KafkaIO y Flink, que es donde el reprocesamiento pasa de verdad.

6.4 Un error que solo una prueba podía encontrar

Vale la pena contarlo porque es el argumento del enunciado comprobado sobre el propio código.

La agregación sumaba el intervalo grosero **junto con** las dos mitades que lo reemplazan: $6 + 2 + 4 = 12$ kWh, el doble del consumo real, sobre un dato que se factura. Se creía que el *upsert* lo resolvía, y no: opera sobre la celda, y los tres intervalos caen dentro de la misma celda (sección 5.4).

Lo que más enseña es que **la primera corrección también estaba mal**, y que tampoco se veía. Descartar los superados en una etapa y sumar en la siguiente vuelve a duplicar, porque dos agregaciones encadenadas bajo `ACCUMULATING` se suman entre sí. Apareció recién al conectar la cadena a Kafka y probarla con una ventana que dispara dos veces.

Con datos ideales ninguno de los dos aparece. El primero necesita una lectura tardía; el segundo, además, que la ventana dispare más de una vez.

6.5 Las irregularidades, a pedido y en vivo

La demostración de la sección 6.1 corre sobre `DirectRunner`, y el recorrido de la sección 6.3 siembra siempre las mismas cinco lecturas. Faltaba poder **provocar** una irregularidad sobre el stack real y ver la respuesta en el momento, que es lo que pide la demostración en vivo. Para eso, el tablero trae un **inyector**: seis botones que publican al tópico crudo una lectura del medidor MED-DEMO-001.

Botón	Qué tiene que pasar
Lectura normal	Aparece o crece la celda de la franja que corresponde
Duplicado de publicación	Nada cambia : mismo <code>event_id</code> , lo descarta el deduplicador
Tardía sobre el borde	Corrige el reparto sin mover el total ; la celda pasa a <code>medida</code>
Reseteo de contador	A cuarentena, con motivo <code>contador_retrocede</code>
Trama truncada	A cuarentena, o un valor absurdo según dónde caiga el corte
Hueco largo	Celda <code>indeterminada</code> , con <code>minutos_indeterminados > 0</code>

El duplicado no se imita, se produce. Como `event_id` es `sha256("<medidor_id>|<instante_lectura>")[:16]`, republicar el mismo par da el mismo identificador por construcción: es un reintento de publicación de verdad. Si el botón generara un evento nuevo, el deduplicador lo dejaría pasar y el total se movería, que es lo contrario de lo que la demostración afirma. Dos pruebas lo fijan.

La tardía necesitó un desvío deliberado. El consumo del inyector es lineal, así que interpolar el cruce de las 18:00 acertaba exacto y la lectura tardía solo cambiaba la etiqueta. Con 0,200 kWh de desvío respecto de la recta modela lo que pasa de verdad: la interpolación supone potencia constante, y en el borde de punta la potencia no es constante. Verificado de punta a punta sobre Kafka y Flink el 30/09. La secuencia `normal` → `normal` → `normal` → `duplicado` → `tardía` produjo cinco revisiones para seis mensajes publicados:

#	Celda	kWh	Origen
1	MED-DEMO-001 2026-09-30 resto	0,900	medida
2	MED-DEMO-001 2026-09-30 resto	1,200	interpolada
3	MED-DEMO-001 2026-09-30 punta	0,600	interpolada
4	MED-DEMO-001 2026-09-30 resto	1,000	medida
5	MED-DEMO-001 2026-09-30 punta	0,800	medida

El duplicado no produjo ninguna revisión. La tardía mueve **0,200 kWh de resto a punta** con el total quieto en 1,800 kWh, y las dos celdas pasan de *interpolada* a *medida*. Es el mismo fenómeno de la sección 6.1, ahora sobre el stack real y a pedido.

6.6 Reproducibilidad: seis corridas, cuatro máquinas

Corrida	Entorno	Qué aportó
Sergio	Máquina de desarrollo	La corrida de referencia, regenerable con <code>evidencia/generar-evidencia.sh</code>
Daniel	Windows + WSL2 Ubuntu 24.04 + Docker Desktop	Mismos números con otras versiones de Docker, Compose y uv; encontró el <i>segfault</i> de <code>uv run pytest</code> , que dio origen al perfil <code>pruebas</code>
Clara	Un tercer entorno	Recorrido completo, los dos replays y la suite dentro del contenedor
Francisco, vueltas 2, 3 y 4	Ubuntu 24.04, persona ajena al equipo	Siguiendo solo el <code>README.md</code> desde el <code>git clone</code> . La primera vuelta no quedó transcrita; sus correcciones están en el historial

Las seis dan **5,500 kWh**, con 2,400 en `resto` y 3,100 en `punta`, y la cuarentena vacía. Están en `evidencia/` con su fecha, su commit y las versiones del entorno.

La verificación externa fue la que más encontró, y siempre lo mismo. El enunciado pide que alguien ajeno al equipo pueda levantar el sistema siguiendo solo el `README`. En cuatro vueltas aparecieron tres defectos, y los tres tenían la misma forma: **lo que probaba nuestra automatización y lo que mandaba hacer el `README` no eran el mismo camino**.

Vuelta	Qué falló	Por qué no lo vimos antes
1. ^a	El perfil <code>demo</code> moría al arrancar: se llamaba como método a una propiedad	Las pruebas cubrían otro punto de entrada. Ahora hay tres pruebas del arranque
3. ^a	El job server moría por OOM en la prueba de humo (sección 2.2)	<code>generar-evidencia.sh</code> no deja el pipeline de streaming corriendo; el <code>README</code> sí
4. ^a	La suite dentro del contenedor no encontraba <code>tablero/</code>	En el host pasaba; el servicio <code>pruebas</code> no montaba la carpeta y su imagen estaba cacheada

La segunda vuelta salió limpia, y aun así la tercera, sobre un clon nuevo, falló. **Una verificación que sale bien una vez no prueba que el camino esté sano**, y la memoria del job server fallaba unas veces sí y otras no. La cuarta vuelta confirmó el arreglo: prueba de humo con 40 lecturas de entrada y 40 de salida, el job server sin reinicios, y las 116 pruebas en verde dentro del contenedor.

7. Límites, supuestos y posibles mejoras

7.1 Supuestos declarados

Supuesto	En qué se apoya
No hay compra de energía al usuario	Es un rasgo del marco regulatorio modelado, no una simplificación nuestra. Sin inyección remunerada, 15.8.0 equivale a energía consumida
Un consumo negativo es siempre un reseteo	Se desprende del anterior: sin exportación, el contador no baja
No rige horario de verano	Evita días con franjas de duración distinta. Si volviera, la configuración tendría que contemplarlos
Ningún medidor acumula por franja	No es una simplificación: es incompatible con que las franjas sean configurables, porque un cambio de calendario obligaría a reconfigurar el parque en campo

7.2 Límites conocidos

La tasa de fallas no está calibrada. Las mediciones disponibles muestran la *forma* de cada caso —la distribución de duraciones es trimodal, la escalera de reintentos es determinista— pero **no permiten estimar con qué frecuencia** ocurre cada uno. Es el parámetro más influyente del modelo, y el error de atribución que el sistema reporta depende de él.

La correlación por modelo de equipo es invisible para este pipeline. Con el mismo enlace, las tasas de entrega por modelo van de 65 % a 94 %, y como los modelos están repartidos por todo el parque esa correlación es **especialmente dispersa**: ninguna partición por ubicación la aísla. Un pipeline particionado por medidor o por cabina no puede verla. Es el límite más interesante que encontramos, porque hay una estructura real en los datos que la clave de particionamiento elegida no alcanza.

Un evento perdido arruina dos intervalos, no uno. Si falta la lectura de las 18:15, no se puede calcular el consumo de 18:00–18:15 ni el de 18:15–18:30: el primero pierde su final y el segundo su inicio.

Un pane por evento tardío. La ráfaga de una cabina que vuelve de una caída produce una escritura por lectura. En producción convendría agrupar los disparos tardíos, a costa de demorar la corrección unos minutos — algo que a la facturación no le cambia nada. Se eligió la versión por evento **para que la corrección sea visible en la demostración**.

Fuera de las 36 horas no hay deduplicación, como se explica en la sección 5.5.

El consumidor confirma offsets por su cuenta. `ReadFromKafka` corre con `enable.auto.commit=true`, y ese commit periódico no espera a que el elemento termine de procesarse. Con checkpoints que completaran no importaría, porque los offsets que valdrían serían los del checkpoint; como no completan (límite siguiente), un trabajo relanzado arranca desde el último offset confirmado, que puede estar por delante de lo procesado, y ahí el tramo de entrada deja de ser «al menos una vez». La corrección es `commit_offset_in_finalize=True`, que confirma recién al cerrar el checkpoint — pero sin checkpoints que cierren, tampoco alcanzaría por sí sola.

Ningún checkpoint completa, así que el estado no sobrevive a un reinicio. Es el límite más serio que encontramos, y lo encontramos tarde: el 30/09, midiendo otra cosa. La configuración está puesta —`execution.checkpointing.interval: 30s`, modo `EXACTLY_ONCE`— y Flink los dispara, pero **ninguno cierra**: sobre un trabajo que llevaba veinte mil mensajes de salida, la API de Flink reportaba `total=4`, `completed=0`, `failed=3`, con **3 de 13 subtareas** confirmando y el resto venciendo por tiempo.

La causa probable es concreta y está a una línea: `esqueleto.opciones` no le pasa `--checkpointing_interval` a Beam, y en Beam el checkpointing está **deshabilitado por defecto** — lo que diga `flink-conf.yaml` configura a Flink, no a los operadores que Beam genera. No se aplicó porque se detectó el mismo día de la entrega y habría invalidado las seis corridas de evidencia ya tomadas; verificarlo de verdad exige repetirlas.

Qué significa en la práctica: el pipeline procesa y emite correctamente —eso está verificado seis veces—, pero **si el trabajo se reinicia, el estado por medidor se pierde**. Las lecturas siguientes no tienen contra qué restarse hasta que llegue la próxima de cada medidor, y la deduplicación olvida lo que había visto. Para una demostración no cambia nada; para producción es lo primero que habría que cerrar.

Con el tópico casi vacío, la lectura no acotada no entrega. Con el pipeline en modo no acotado y muy pocos mensajes en el tópico, las lecturas se consumen —el grupo queda con lag 0— pero no salen de la etapa de lectura de `KafkaIO`: el vértice `LeerLecturas` recibe registros y emite cero. En cuanto hay volumen, las mismas lecturas salen con los valores esperados. No afecta a `humo` ni a `extremo_a_extremo`, que leen acotado, ni al perfil `demo`, donde el simulador publica miles de lecturas. Sí afecta al inyector: **para usarlo en vivo hay que dejar el simulador corriendo**. La causa no está aislada.

Las métricas existen, pero no se ven en Flink, como explica la sección 2.3. Para observar el trabajo mientras corre hoy hay que ir a la interfaz de Flink, a los offsets de Kafka o al tablero.

El job server acumula memoria entre trabajos. Con `1800m` alcanza para el recorrido completo del README, que sirve cuatro trabajos. Una sesión más larga sin reiniciar el stack podría volver a agotarlo.

7.3 Posibles mejoras

Cerrar los checkpoints, antes que cualquier otra cosa. Pasar `--checkpointing_interval` en `esqueleto.opciones`, comprobar con la API de Flink que completan, y recién ahí cambiar el consumidor a `commit_offset_in_finalize=True`. Hay que repetir las corridas de evidencia después de hacerlo, y por eso no entró en esta entrega.

Un *metrics reporter* en Flink, para que los contadores de la sección 2.3 se vean en la interfaz mientras el trabajo corre. Es configuración de `flink-conf.yaml`, no código.

Un dispositivo de lectura por medidor. Elimina de raíz la limitación de fondo, que es el bus compartido. Y su justificación económica **sale de este mismo trabajo**: el error de atribución que el pipeline mide hoy es exactamente lo que esa inversión ahorraría. El simulador genera ese escenario y el **mismo pipeline, sin cambios**, lo procesa — así la comparación es una medición y no una estimación.

Si se hace, conviene que el dispositivo exponga sus parciales en **códigos OBIS propios** y no en los registros tarifarios estándar: permite leer el total del medidor y los parciales del dispositivo y **verificar que sumen**, de modo que un dispositivo desincronizado se detecte solo.

Reducir la pausa entre medidores. Un tercio del tiempo por medidor es una pausa fija que el concentrador aplica, y es una decisión de implementación, no del protocolo. Es el parámetro más barato de mejorar de todos los que aparecen en este análisis.

Medición neta. Si se introdujera compra de energía al usuario, habría que leer `1.8.0` y `2.8.0` por separado y la regla del consumo negativo dejaría de valer.

Una **materialización intermedia** para el tablero, si alguna vez hacen falta consultas ad-hoc o muchos lectores concurrentes.

8. Integrantes y contribuciones

Integrante	Contribución principal
Sergio Morel	Simulador : datos sintéticos deterministas por semilla, con inyección deliberada de fallas —pedidos corridos, cabinas caídas, tramas truncadas, duplicados y ráfagas tardías—. Infraestructura : Kafka, Flink, el job server y la resolución de <code>KafkaIO</code> como transformación <i>cross-language</i> . Cadena del pipeline : deduplicación con estado y timer, diferenciación del contador acumulado, y la etapa de celdas vigentes. Franjas : calendario tarifario, atribución y reparto de un intervalo entre los bordes que cruza, con el umbral de 90 min elegido sobre la distribución medida. 92 de las 116 pruebas , la demostración narrada, las dos verificaciones del recorrido completo, el replay desde el offset 0 y el DAG dibujado desde el código. Las correcciones que dejó la verificación externa: el arranque del perfil <code>demo</code> , la memoria del job server y el montaje del perfil <code>pruebas</code> . Secciones 1, 2, 5, 6 y 7 de este documento. Encontró los dos errores de doble conteo de sección 5.4
Clara Almirón	Contratos de evento de entrada y de salida : clave de particionamiento, cantidad de particiones, tópicos, regla de versionado y política de cuarentena. Política temporal : ventana diaria alineada al día local y lateness de 36 h, con su justificación contra 24 y 48. Tablero en marimo que cierra el ciclo del dato, y su inyector de irregularidades , con sus 24 pruebas y la verificación de punta a punta sobre el stack. Métricas del camino caliente (sección 2.3) y la corrección posterior de que no se ven en Flink. Garantías del productor de salida — <code>acks=all</code> e idempotencia—, verificadas contra el stack. Una corrida completa de evidencia en un tercer entorno. Tablero de pendientes y mapa de la documentación , que son la convención con que el equipo se mantuvo sincronizado. Revisión de las secciones 3 y 4 de este documento, donde detectó que la política de <i>triggers</i> documentada no existía en el código
Daniel Ramírez	Validación de reproducibilidad en plataforma independiente . Ejecutó el recorrido completo en una segunda máquina —Windows + WSL2 Ubuntu 24.04 + Docker Desktop, con otras versiones de Docker, Compose y <code>uv</code> — y obtuvo los mismos resultados, lo que convierte la reproducibilidad en algo verificado y no afirmado. Encontró además que <code>uv run pytest</code> aborta con <i>segmentation fault</i> en ese entorno, hallazgo que derivó en el perfil <code>pruebas</code> que corre la suite dentro del contenedor. Su evidencia está en <code>evidencia/</code>

`git shortlog -sn --no-merges` respalda las contribuciones **de código**. La de Daniel no es de código sino de validación, y su evidencia está en el repositorio: una corrida completa del recorrido en otra máquina, con su fecha, su commit y las versiones del entorno.

Francisco, que no integra el equipo, hizo la verificación externa que pide el enunciado: levantó el sistema en cuatro vueltas siguiendo solo el `README.md`, y encontró tres de los defectos de la sección 6.6.